Arbitrary Arithmetic Library: Project Report

DIVYAANJALI C, ID: CS24BTECH11019

May 2, 2025

Contents

1 NAME/ID

This project was DONE BY:

- DIVYAANJALI C, ID: CS24BTECH11019

The Arbitrary Arithmetic Library was created as part of a Software Development course to support arithmetic operations with very large numbers in Java. The project features a library for both integer and floating-point calculations, along with a command-line interface and helper scripts for compiling and running the code.

2 INTRODUCTION

The Arbitrary Arithmetic Library is a Java-based project built to handle arithmetic operations on very large integers and high-precision decimals—beyond what Java's built-in types like long and double can manage. It uses string-based representations to perform addition, subtraction, multiplication, and division with arbitrary precision.

The project includes three main components: AInteger for large integer operations, AFloat for floating-point numbers, and MyInfArith, a command-line tool for running the library. It also includes scripts for easy compilation (via Ant) and execution (with Python). Git was used for version control and tracking progress through milestones.

3 STRUCTURE

The Arbitrary Arithmetic Library is a Java-based library designed to perform arbitrary-precision arithmetic operations on integers and floating-point numbers. It consists of three main classes in the `arbitraryarithmetic` package: `AInteger`, `AFloat`, and `MyInfArith`, supported by an Ant build script (`build.xml`) and a Python runner script (`run.py`). This section details the design decisions, including class structure, algorithm choices, supporting scripts, package organization, and UML class diagrams.

3.1 Class Structure

The library is structured around three classes:

- `AInteger`: Implements integer arithmetic with arbitrary precision by storing numbers as strings, allowing the handling of values much larger than Java's long can support. The methods include:
  - `add(AInteger)`: Adds two integers digit-by-digit with carry handling.
  - `subtract(AInteger)`: Subtracts one integer from another, managing borrow.
  - `mul(AInteger)`: Multiplies two integers using partial products.
  - `div(AInteger)`: Performs integer division via repeated subtraction.
  - `compare(String, String)`: Compares two numbers for ordering.
  - `removeZeroes(String)`: Removes leading zeros from a string representation.
  - `parse(String)`: Converts a string to an `AInteger` object.
  - `toString()`: Returns the string representation of the number.

  Example `removeZeroes` implementation:

```java
public String removeZeroes(String num) {
    if (num.startsWith("-")) {
        return "-" + num.substring(1).replaceFirst("^0+", "");
    }
    return num.replaceFirst("^0+", "");
}
```

- `AFloat`: Builds on the integer arithmetic logic to support floating-point numbers, handling decimal points and maintaining precision throughout calculations. Like AInteger, it uses strings to represent numbers, enabling accurate operations on values with fractional parts. AFloat can perform high-precision arithmetic—supporting up to 30 decimal places for division—and is designed to manage complex floating-point operations. Key methods include:
  - `AFloat()`: Default constructor initializing an empty floating-point number.
  - `AFloat(String)`: Constructs an `AFloat` from a string (e.g., `"123.456"`).
  - `AFloat(AFloat)`: Copy constructor for creating a new `AFloat` from an existing one.
  - `parse(String): AFloat`: Parses a string to create an `AFloat` object, validating decimal points and digits.
  - `getString(): String`: Returns the internal string representation.
  - `toString(): String`: Returns a formatted string of the number (e.g., `"123.456"`).
  - `add(AFloat): AFloat`: Adds two floating-point numbers by aligning decimal points and using integer addition.
  - `subtract(AFloat): AFloat`: Subtracts by aligning decimals and using integer subtraction.
  - `mul(AFloat): AFloat`: Multiplies by adjusting decimal positions and using integer multiplication.
  - `div(AFloat): AFloat`: Divides with configurable precision (default 30 decimals), handling quotient and remainder.

  Example `getString` implementation:

```java
public String getString() {
    return this.value;
}
```

- `MyInfArith`: A command-line tool that lets users run the library from the terminal. It takes four inputs: the type of number (`int` or `float`), the operation (`add`, `sub`, `mul`, or `div`), and the two numbers to use. It then calls either `AInteger` or `AFloat` to do the calculation, depending on the type. This makes it easy to test the library without writing extra Java code. **Error Handling in MyInfArith**: The `MyInfArith` class includes error handling for invalid inputs. If fewer than four arguments are provided, it outputs: `arguments are less please check your arguments`. For invalid types (not `int` or `float`), it outputs: `Invalid type: <type>. Must be 'int' or 'float'`. For invalid operations (not `add`, `sub`, `mul`, or `div`), it outputs: `Invalid operation: <operation>`. If a calculation error occurs (e.g., division by zero), it outputs: `Error during calculation: <error`$_{message}$ $>$.

3.2 Algorithm Choices

Arithmetic operations are implemented using digit-by-digit algorithms:

- **Addition and Subtraction**: Process digits from right to left, handling carry (addition) or borrow (subtraction). For example, adding `"123"` and `"456"` involves aligning digits, summing with carry, and producing `"579"`.
- **Multiplication**: Uses a grade-school algorithm, computing partial products for each digit pair and summing them with `add`. For `"123"` × `"456"`, partial products are aggregated to produce `"56088"`.
- **Division**: Implements long division by repeatedly subtracting the divisor from a portion of the dividend, counting iterations to form the quotient. For `"132"` ÷ `"22"`, the result is `"6"`.
- **Negative Numbers**: Extracts signs, performs operations on absolute values, and applies sign rules (e.g., negative + negative = negative sum).

explanation: We chose these algorithms because they are easy to understand, work well with very large numbers, and fit naturally with our use of strings to represent values. This approach helps ensure accurate results without depending on Java's built-in number types.

3.3 Arithmetic Operations in AInteger

The `AInteger` class implements arithmetic operations for arbitrarily large integers using string-based digit manipulation. Each operation is designed to handle large numbers without overflow, supporting positive and negative inputs.

- **Addition**: Adds two integers digit-by-digit, processing from right to left with carry handling. If operands have opposite signs, the operation is redirected to subtraction with adjusted signs. The result is normalized (leading zeros removed) and the sign is set based on the operands. For example, adding `"123"` and `"456"` yields `"579"`.
- **Subtraction**: Subtracts one integer from another, handling borrows digit-by-digit. If operands have different signs, subtraction becomes addition. Magnitudes are compared to determine the order of subtraction, and the result is normalized with the correct sign. For example, subtracting `"456"` from `"579"` yields `"123"`.
- **Multiplication**: Uses the schoolbook algorithm, computing partial products for each digit pair and summing them using `add`. Partial products are padded with zeros based on position, and the result is normalized. For example, multiplying `"123"` by `"456"` produces `"56088"`.
- **Division**: Implements long division by repeatedly subtracting the divisor from a running remainder, building the quotient digit-by-digit. If the divisor is zero, an `ArithmeticException` is thrown. For example, dividing `"132"` by `"22"` yields `"6"`.

3.4 Arithmetic Operations in AFloat

The `AFloat` class supports floating-point arithmetic with high precision, leveraging `AInteger` for integer-level operations. It handles decimal alignment, normalization, and signs, supporting up to 30 decimal places for consistency with project requirements.

- **Addition**: Aligns decimal points by padding the shorter operand with zeros. The numbers are treated as integers by removing decimal points, and `AInteger.add` is used. The

result is normalized and adjusted to 30 decimal places. For example, adding `"123.45"` and `"67.89"` yields `"191.34"`.

- **Subtraction**: Similar to addition, aligns decimal points and uses `AInteger.sub` for integer subtraction. The result is normalized to 30 decimal places. For example, subtracting `"67.89"` from `"123.45"` yields `"55.56"`.
- **Multiplication**: Calculates the total decimal places in both operands, removes decimal points, and uses `AInteger.mul`. The result's decimal point is placed based on the total decimal places, and the result is normalized to 30 decimal places. For example, multiplying `"3.14"` by `"2.0"` yields `"6.28"`.
- **Division**: Scales the numerator by 10^{30} to preserve precision, removes decimal points, and uses `AInteger.div`. The result's decimal point is adjusted, and the result is normalized to 30 decimal places. For example, dividing `"10.0"` by `"2.0"` yields `"5.0"`.

3.5 Supporting Scripts

The project includes scripts to streamline compilation and execution:

- `build.xml`: An Ant build script that compiles Java source files into an `out` directory and packages them into a JAR file named `arithmetic.jar`. The script includes three targets:
  - `compile`: Creates the `out` directory and compiles all Java files.
  - `jar`: Packages compiled classes into `arithmetic.jar`.
  - `run`: Executes `MyInfArith` from the JAR with user-provided arguments.
- `run.py`: A Python script that checks for the `out` directory, compiles the project using `ant` if needed, and runs `MyInfArith` with user arguments. Example snippet:

```python
import sys
import os
import subprocess

def compile_run(args):
    if not os.path.exists("out"):
        print("'out' directory not found. Running 'ant'...")
        subprocess.run(["ant"])
    result = subprocess.run(["java", "-cp", "out", "
        arbitraryarithmetic.MyInfArith"] + args, capture_output=
        True)
    print(result.stdout.decode())
```

Explanation: The Ant script automates compilation and packaging, while `run.py` simplifies execution by handling compilation checks and argument passing, improving usability across platforms.

3.6 Package Organization

All classes reside in the `arbitraryarithmetic` package.
explanation: A single package maintains a clear namespace for a small library, with potential for future expansion (e.g., adding modular arithmetic classes).

3.7 UML Class Diagrams

The UML class diagram (Figure 1) illustrates the library's structure.

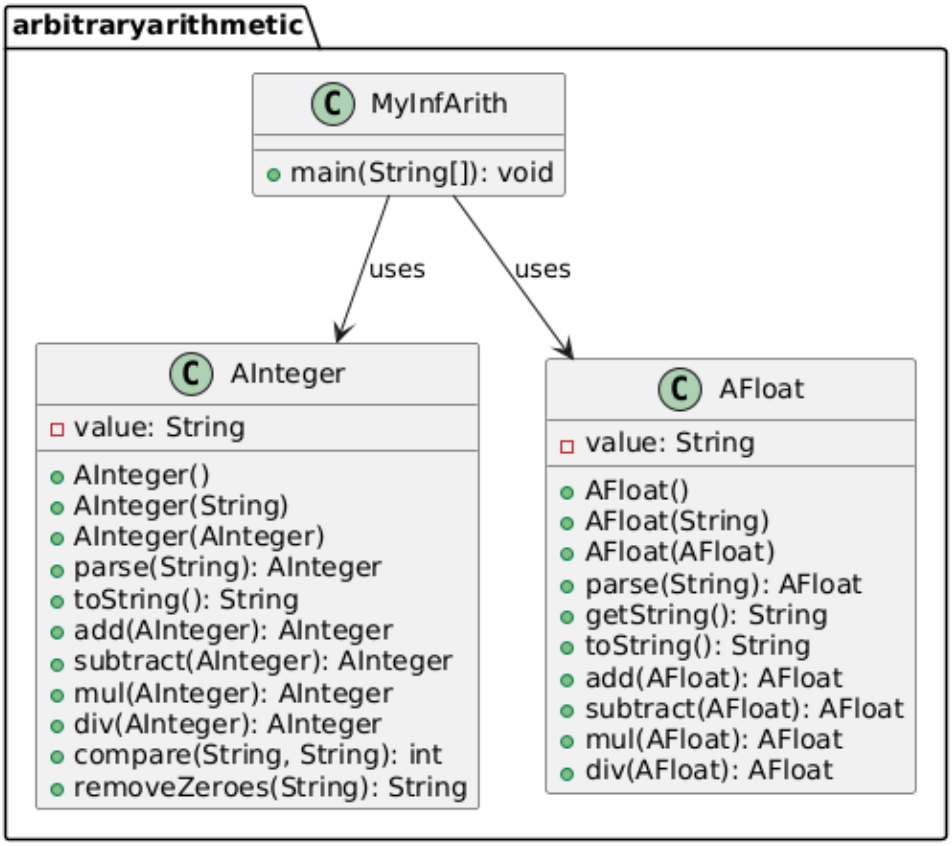

Figure 1: UML Class Diagram for Arbitrary Arithmetic Library

UML Description:

- `AInteger`:
  - **Attributes**: `-value: String`
  - **Methods**:
    - `+AInteger()`
    - `+AInteger(String)`
    - `+AInteger(AInteger)`
    - `+parse(String): AInteger`
    - `+toString(): String`
    - `+add(AInteger): AInteger`
    - `+subtract(AInteger): AInteger`
    - `+mul(AInteger): AInteger`
    - `+div(AInteger): AInteger`
    - `+compare(String, String): int`
    - `+removeZeroes(String): String`
- `AFloat`:

  - **Attributes**: `-value: String`
  - **Methods**:
    * `+AFloat()`
    * `+AFloat(String)`
    * `+AFloat(AFloat)`
    * `+parse(String): AFloat`
    * `+getString(): String`
    * `+toString(): String`
    * `+add(AFloat): AFloat`
    * `+subtract(AFloat): AFloat`
    * `+mul(AFloat): AFloat`
    * `+div(AFloat): AFloat`
- `MyInfArith`:
  - **Methods**: `+main(String[]): void`
- **Relationships**: `MyInfArith` uses `AInteger` and `AFloat` via method calls. No inheritance exists, but the classes share similar interfaces for arithmetic operations.

4 README Section

The `README.md` file in the repository root provides instructions for using the Arbitrary Arithmetic Library. It covers installation, usage, and setup details, including how to compile the project with Ant, run the `MyInfArith` driver using the `run.py` script, and integrate the library into custom Java code. For complete guidance, refer to `README.md`.

5 Git Commits Snapshot

Git was used for version control to track the development of the Arbitrary Arithmetic Library. Commits and tags (e.g., `v1.0-initial`, `v1.3-final`) document the project's progress through milestones, such as the implementation of `AInteger`, `AFloat`, `MyInfArith`, and supporting scripts and documentation.

6 Code Running Process

The Arbitrary Arithmetic Library can be compiled and executed using provided scripts. The Ant build script (`build.xml`) compiles the Java source files into a `build` directory. The Python script (`run.py`) simplifies execution by checking for compilation, then running the `MyInfArith` driver with user-specified arguments for arithmetic operations. Example usage:

```
python3 run.py int add 123 456
```

This produces the output `579`. For detailed instructions, refer to `README.md`.

7 Verification Approach

The implementation was verified through multiple methods to ensure correctness and identify limitations:

- **Manual Testing**: The `MyInfArith` driver was tested with various inputs:
  - Positive numbers: `python3 run.py int add 123 456` (Expected: `579`)
  - Negative numbers: `python3 run.py float sub 45.67 -12.34` (Expected: `58.01`)
  - Zero: `python3 run.py int mul 123 0` (Expected: `0`)
  - Large numbers: `python3 run.py int mul 123456789 987654321`
  - Floating-point: `python3 run.py float mul 3.14 2.0` (Expected: `6.28`)
- **Edge Case Testing**: Tested division by zero (`python3 run.py int div 123 0`), empty inputs, and negative numbers to confirm error handling.
- **Debugging**: Temporary print statements were added to `mul` to verify partial products during development (removed in the final version).
- **Code Review**: The implementation was reviewed for correctness, focusing on sign handling, decimal alignment in `AFloat`, and algorithm accuracy.

8 Conclusion

The Arbitrary Arithmetic Library provides a simple and reliable way to handle arithmetic with very large or very precise numbers, which are too big for Java's regular number types. We built three main parts: `AInteger` for big integers, `AFloat` for decimal numbers, and `MyInfArith` as a command-line tool to use them. These use strings and basic math logic to calculate results digit by digit. We also added helper scripts like `build.xml` and `run.py` to make building and running the code easier. Using Git helped us manage and track our progress. Through this project, we learned a lot about how arithmetic works, how to design clean code using objects, and why it's important to test properly and write good documentation. This library can be useful in any project that needs very accurate math, and it's a good base for adding more features in the future.